

EngageNet

M3 Presentation - May 31, 2026

Lado Turmanidze, Luka Javakhishvili, Mariami Gadelia, Keso Chikhladze

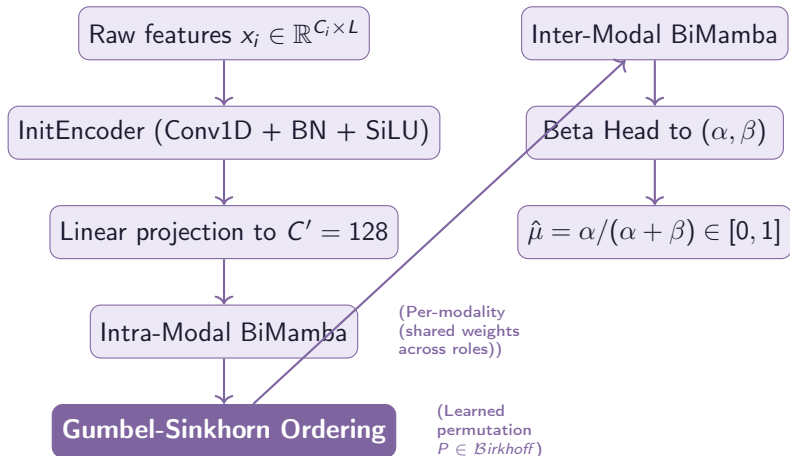
Supervisors: Dr. Philipp Müller & Beso Mikaberidze

May 31, 2026

Today's Agenda

1. **Full architecture recap** - end-to-end data flow
2. **Beta distribution justification** - why not Kumaraswamy or logit-normal?
3. **Implementation** - JAX + Flax Linen, project structure
4. **Engineering beyond the paper** - what research papers never show
5. **Evaluation metrics** - CCC and CDD
6. **Status & next steps**

PART 1: Full Architecture - Data Flow



Architecture - Key Design Choices

Selective State Space (SSM)

- ▶ Input-dependent Δ_t, B_t, C_t
- ▶ $O(\log L)$ via associative scan
- ▶ Bidirectional: forward + reversed

Weight Sharing

Same encoder for expert & novice of the same modality $\rightarrow$ 10 encoders, not 20.

Inter-Modal Fusion

- ▶ Transpose: time $\leftrightarrow$ modality
- ▶ BiMamba across $\sum C'_i$ axis
- ▶ Order learned via Gumbel-Sinkhorn

Core Modalities (4)

FAUs, gaze, pose, audio (eGeMAPS). Chosen for temporal richness & complementarity.

PART 2: Why Beta? - Comparison with Alternatives

Requirements

1. Support on $[0, 1]$ (no leakage)
2. Closed-form variance at inference
3. Closed-form KL for TTA loss
4. Differentiable for backprop

Scorecard

	Beta	Kuma.	Logit-N
Support $[0, 1]$	✓	✓	✓
Var (closed)	✓	×	×
KL (closed)	✓	×	~
Fast inference	✓	×	×
Fast training	×	✓	✓

Beta vs Kumaraswamy - The Inference Cost

Kumaraswamy variance

Requires Γ function evaluations:

$$\text{Var}[Y] = b B(1 + \frac{2}{a}, b) - [b B(1 + \frac{1}{a}, b)]^2$$

Expensive at 25 Hz real-time streaming.

Monotonic proxy: $\partial U / \partial \kappa < 0$ and $\partial H / \partial \kappa < 0 \implies$ variance ranks samples identically to differential entropy.

Beta variance

Pure arithmetic:

$$U(x) = \frac{\alpha\beta}{(\alpha+\beta)^2(\alpha+\beta+1)}$$

No transcendental functions at inference.

Beta vs Logit-Normal - The KL Problem

Logit-Normal issues

- ▶ Moments need numerical integration
- ▶ KL in $[0, 1]$ space: no closed form
- ▶ Gradients explode near boundaries

Beta KL (closed-form)

$$D_{KL}(B_1 \| B_2) = \ln \frac{B_2}{B_1} + (\alpha_1 - \alpha_2) \psi(\alpha_1) + \dots$$

Exact, stable, used directly in $\mathcal{L}_{\text{mis}}$ for TTA.

Beta: transcendental cost during training (digamma in grads), free at inference.

PART 3: Implementation - JAX + Flax Linen

Why JAX?

- ▶ `jax.lax.associative_scan` for $O(\log L)$ SSM
- ▶ XLA compilation
- ▶ Functional transforms: `jit`, `grad`
- ▶ I wanted to see what the hype was about

Why Flax Linen?

- ▶ `nn.Module` with explicit params
- ▶ Kinda like functional programming (I love OCaml)
- ▶ Seamless Orbach checkpointing
- ▶ Clean separation: model vs state

All randomness controlled via `jax.random.PRNGKey(95)` with explicit key splitting.

Project Structure

src/ - Core modules

<code>config.py</code>	Centralised cnfg
<code>ssm.py</code>	JAX assoc. scan
<code>bimamba.py</code>	BiMambaBlock
<code>inter_modal.py</code>	Gumbel-Sinkhorn
<code>beta_head.py</code>	Beta regression
<code>model.py</code>	Full EngageNet
<code>init_encoder.py</code>	Shallow CNN
<code>modality_frontend.py</code>	Per-modality entry

src/ - Pipeline & eval

<code>train.py</code>	AdamW + early stop
<code>tta.py</code>	Test-time adaptation
<code>inference.py</code>	Multi-corpus CSV out
<code>dataset.py</code>	Lazy streaming loader
<code>data_loader.py</code>	Batch iterator
<code>metrics.py</code>	CCC, CDD
<code>evaluate.py</code>	Scoring script
<code>aggregator.py</code>	Overlap-add windows

PART 4: What We Added Beyond the Paper

Data engineering?

- ▶ Lazy streaming for 88+ GiB dataset (one session at a time in memory)
- ▶ Dynamic `active_modalities` filter

Reproducibility

- ▶ CLI via `EngageNetConfig.from_cli()`
- ▶ Orbax checkpoints every N epochs
- ▶ Docker containers (train + inference)

Training details

- ▶ Tau annealing: $\tau_t = \max(\tau_\infty, \tau_0 \cdot \gamma^t)$
- ▶ CCC-based validation + early stopping
- ▶ AdamW with weight decay

Inference pipeline

- ▶ Per-session processing
- ▶ Overlap-add window aggregation
- ▶ Submission CSV generation

Surgical TTA - Implementation Choices

Frozen vs updated

- ▶ **Updated:** BatchNorm, first Conv1D in InitEncoder, first Dense in inter-modal
- ▶ **Frozen:** All SSM params, Beta head, intra-modal BiMamba weights

Why surgical?

- ▶ Prevents catastrophic forgetting
- ▶ Mitigates Variance Starvation: model cannot inflate $\text{Var}[y]$ to bypass hard samples
- ▶ Only 3–5% of total params updated

PART 5: Evaluation Metrics

CCC (primary metric)

$$\text{CCC} = \frac{2\rho\sigma_{\hat{y}}\sigma_y}{\sigma_{\hat{y}}^2 + \sigma_y^2 + (\mu_{\hat{y}} - \mu_y)^2}$$

Penalises both scale and location shift.

Combined: $\frac{1}{N} \sum_n \text{CCC}_n$.

CDD (fairness)

$$\text{CDD} = \frac{1}{|G|} \sum_g |\text{CCC}_g - \text{CCC}_{\text{all}}|$$

Computed per gender and per language.

Lower = more equitable.

Challenge ranking: maximise CCC while keeping CDD below threshold.

Claude Sonnet 4.6 was used to code the following files (I was too lazy to write them myself):

- ▶ All 3 test files: `tests/test_frontend.py`, `tests/test_model.py` and `tests/test_inter_modal.py`
- ▶ Forgot to include CCC, and I wanted to reach a milestone before `git push`: `src/evaluate.py`
- ▶ I did not figure out a neat way to do the stuff: `surgical_mask` method in `src/tta.py`

Done ✓

- ▶ Full architecture coded
- ▶ All unit tests pass
- ▶ Docker containers ready
- ▶ Paper draft in solid shape

In Progress & Next Steps

- ▶ Using MICM compute for training
- ▶ Training on NoXi
- ▶ Hyperparameter tuning?
- ▶ TTA evaluation on NoXi+J

Questions?